Bases de Datos Entrega Final

Fabian Andres Rincon Rincon
Facultad de Ingeniería
Programa de Ingeniería Electrónica extensión Tunja
Universidad Pedagógica y Tecnológica de Colombia
Tunja, Boyacá - Colombia
fabian.rincon04@uptc.edu.co

***Resumen*—Este informe documenta el diseño, desarrollo e implementación de un Sistema Integral de Dashboard para la base de datos PEDIDOS, que unifica visualización interactiva, análisis estadístico avanzado y tratamiento automatizado de datos en una plataforma web unificada. El sistema aborda la necesidad crítica de transformar datos transaccionales empresariales en insights accionables mediante una arquitectura Three-Tier basada en Flask (Python), MySQL y tecnologías web modernas.**

La solución implementa 22 consultas estratégicas organizadas en cuatro módulos principales: Dashboard para métricas comerciales en tiempo real, Estadísticas para análisis descriptivos avanzados, Tablas para exploración directa de datos, y Tratamiento para preprocesamiento automatizado aplicando técnicas de imputación, detección de outliers, codificación de variables categóricas y normalización. Cada módulo responde a necesidades específicas del negocio, desde el monitoreo operacional hasta la preparación de datos para machine learning.

Palabras Clave: Business Intelligence, Dashboard Interactivo, Análisis de Datos, Preprocesamiento, Flask, MySQL, Visualización, Sistema Three-Tier, Toma de Decisiones.

I. Introducción

En la era actual de la transformación digital, las organizaciones se enfrentan al desafío constante de gestionar y analizar grandes volúmenes de información. La base de datos PEDIDOS, objeto de este estudio, representa un sistema transaccional crítico que almacena información fundamental sobre operaciones comerciales, gestión de empleados, control de inventario y seguimiento de clientes. Sin embargo, la calidad de los datos constituye un prerrequisito indispensable para extraer insights valiosos y tomar decisiones empresariales fundamentadas.

El preprocesamiento de datos emerge como una etapa crucial en el ciclo de vida analítico, donde la calidad, consistencia y confiabilidad de la información determinan directamente la efectividad de los modelos predictivos y análisis posteriores. Según estudios especializados, aproximadamente el 60-80% del tiempo en proyectos de ciencia de datos se dedica a la preparación y limpieza de datos, evidenciando la importancia crítica de este proceso..

II. Marco teórico del tema principal del proyecto.

El proyecto se enmarca en la intersección de dos áreas fundamentales de la ingeniería de software y la ciencia de datos: la Analítica de Datos y el Desarrollo de Aplicaciones Web Dinámicas. A continuación, se describen los pilares teóricos que sustentan el desarrollo de la aplicación.

1. Fundamentos de Bases de Datos Relacionales

El proyecto se basa en la gestión de datos estructurados mediante una Base de Datos Relacional (RDBMS) (pedidos11).

- Modelo Relacional: Se utiliza el Modelo Relacional propuesto por E.F. Codd, donde los datos se organizan en tablas (relaciones) compuestas por filas (tuplas) y columnas (atributos).

- Integridad y Consistencia: La consistencia de los datos se garantiza a través de las Restricciones de Integridad Referencial (Claves Primarias y Claves Foráneas), que aseguran que las relaciones entre entidades, como la de Pedidos a Clientes, sean válidas y coherentes.

- Lenguaje SQL: El Structured Query Language (SQL) es el lenguaje estándar utilizado para definir, manipular y consultar los datos (CRUD - Crear, Leer, Actualizar, Borrar). Las consultas avanzadas (JOIN, GROUP BY, COUNT) son esenciales para la extracción de las métricas que alimentan el dashboard.

2. Arquitectura Web y Backend con Python/Flask

La implementación del sistema se realiza bajo una arquitectura web modular y ligera, donde Python actúa como el núcleo de la lógica de negocio.

- Micro-framework Flask: Se utiliza Flask, un micro-framework de Python, para construir el backend. Flask facilita la creación de rutas y el manejo de solicitudes HTTP, siendo el pilar para la construcción de la API.

- API RESTful: La aplicación expone endpoints de API (ej. /api/datos-grafico) que siguen, de manera simplificada, los principios REST (Representational State Transfer). El propósito es servir los datos procesados por Python al frontend en formato JSON (JavaScript Object Notation), el formato estándar para el intercambio de datos en la web.

- Entorno Virtual (venv): El uso de un Entorno Virtual asegura que las dependencias del proyecto (pip install...) estén aisladas del sistema operativo principal, garantizando la portabilidad y la consistencia del entorno de desarrollo

3. Visualización de Datos y Frontend

El componente visual es clave para la comunicación efectiva de los insights analíticos.

- Separación de Preocupaciones: La aplicación implementa el principio de Separación de Preocupaciones, donde el frontend (HTML/CSS/JavaScript) se encarga de la presentación, y el backend (Python/Flask) se encarga de la lógica y el procesamiento de datos.

- Visualización Dinámica: Se utilizan librerías de JavaScript como Chart.js para renderizar los gráficos de forma dinámica. El código de JavaScript consume el JSON proporcionado por la API de Flask y lo traduce en gráficos interactivos (barras, áreas, pastel) para la pestaña "Dashboard" y "Estadísticas".

- Dashboard: Es una interfaz visual que presenta de forma consolidada y legible los Indicadores Clave de Rendimiento (KPIs) y las principales conclusiones del análisis de datos.

III. Desarrollo de la práctica y analisis

1. Origen de dataset

El proyecto utiliza la base de datos PEDIDOS, una base de datos transaccional empresarial que gestiona información integral sobre operaciones comerciales. La base de datos incluye múltiples entidades interrelacionadas que capturan el ciclo completo de negocio:

- Gestión de Clientes: Información demográfica y de contacto

- Control de Productos: Inventario, categorías y precios

- Procesamiento de Pedidos: Transacciones comerciales completas

- Administración de Empleados: Recursos humanos y estructura organizacional

- Gestión Geográfica: Ubicaciones y territorios comerciales

Ya los datos suministrados, tablas y demás, fueron tomados de la base de datos realizada en trabajos anteriores

2. Bases de datos de MySQL.

El esquema de base de datos pedidos11 implementa un modelo relacional normalizado que estructura la información empresarial en ocho tablas interconectadas (categorias, clientes, departments, employees, lugares, productos, pedidos, detalles_pedidos), estableciendo mediante 14 restricciones de clave foránea un sistema integral de integridad referencial que garantiza la consistencia de los datos transaccionales. Este diseño optimizado, que incluye estrategias de recreación controlada (DROP TABLE IF EXISTS / CREATE TABLE IF NOT EXISTS), soporta eficientemente las 22 consultas analíticas del dashboard, facilitando tanto la gestión operacional como el análisis empresarial avanzado mediante relaciones bien definidas entre entidades comerciales, recursos humanos, transacciones y estructura geográfica.

3. Implementación de dashboard desde python

Como primer paso se procedió con la creación de una carpeta para albergar todos los archivos a emplear en este dashboard. Inicialmente se desarrolló un código de prueba para verificar la conectividad con la base de datos MySQL, específicamente con la base pedidos11 alojada en el puerto 3308. Esta validación inicial fue crucial para asegurar que la infraestructura de datos estuviera accesible antes de proceder con el desarrollo de la aplicación Flask. Una vez confirmada la conexión exitosa, se estructuró el proyecto en una arquitectura modular organizada en las siguientes carpetas: templates para los archivos HTML del frontend, static para los recursos CSS, JavaScript e imágenes, y fpreprocesamiento para los módulos Python de procesamiento de datos. Esta organización facilitó el mantenimiento del código y permitió una clara separación de responsabilidades entre la lógica de presentación, los estilos visuales y la manipulación de datos.

Para establecer la conexión, fue esencial crear un entorno virtual usando Python. Este entorno se generó dentro de la carpeta del proyecto mediante el comando python -m venv venv. Para permitir que la shell de PowerShell ejecute scripts localmente, se empleó la política de ejecución Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process. Finalmente, para activar el entorno virtual y aislar las dependencias del proyecto, se utilizó el comando específico del sistema, el cual no fue completado en la descripción, pero comúnmente es: .\venv\Scripts\activate, como se visualiza en la imagen

```
PS C:\Users\Andrés\Desktop\Bases_datos_pedidos> .\venv\Scripts\activate
```

Fig. 1. Activación del entorno virtual

Para gestionar e instalar estas dependencias, se utilizó el gestor de paquetes de Python, pip, seguido de la lista de librerías requeridas. El comando general empleado fue: pip install <nombre_librería_1> <nombre_librería_2> .... Es crucial que esta instalación se realice mientras el entorno

virtual (venv) esté activo para asegurar que las librerías se instalen de forma aislada dentro de dicho entorno y no globalmente en el sistema. Al ejecutar el comando pip list se observa algunas librerías instaladas

```
(venv) PS C:\Users\Andrés\Desktop\Bases_datos_pedidos> pip list
Package         Version
--------------- -----------
blinker         1.9.0
click           8.3.0
colorama        0.4.6
contourpy       1.3.3
cycler          0.12.1
Flask           3.1.2
fonttools       4.60.1
itsdangerous    2.2.0
Jinja2          3.1.6
joblib          1.5.2
kiwisolver      1.4.9
MarkupSafe      3.0.3
matplotlib      3.10.7
numpy           2.3.4
packaging       25.0
pandas          2.3.3
patsy           1.0.2
pillow          12.0.0
pip             25.3
PyMySQL         1.1.2
pyparsing       3.2.5
python-dateutil 2.9.0.post0
pytz            2025.2
scikit-learn    1.7.2
scipy           1.16.3
```

Fig. 2. Librerías instaladas.

La estructura del proyecto segui una convención estándar para aplicaciones web, utilizando Flask o un framework similar, evidenciado por la presencia de app.py y la carpeta templates.

Estructura General del Proyecto

El directorio raíz, llamado BASES_DATOS_PEDIDOS, contiene varios elementos clave. Incluye el archivo principal de la aplicación, app.py, que presumiblemente gestiona la lógica de backend y el enrutamiento. También se observa la carpeta venv, que contiene el entorno virtual de Python, aislando las dependencias del proyecto. Además, existe una carpeta fpreprocesamiento que incluye archivos de preprocesamiento de datos, como preprocesamiento.py y los directorios __pycache__, indicando la presencia de módulos y paquetes de Python.

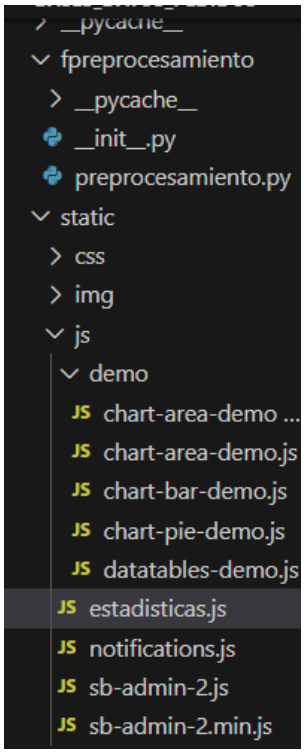

Fig. 3. Estructura General del Proyecto

Contenido Estático y Vistas

Para la interfaz de usuario, el proyecto se divide en las carpetas static y templates.

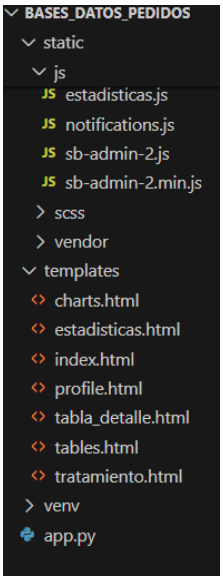

Fig. 4. Contenido estático y vistas

La carpeta templates almacena las vistas HTML de la aplicación, como index.html, charts.html, tables.html, y otras específicas como estadisticas.html y tratamiento.html, que

definen el diseño y la estructura del contenido que se muestra al usuario.

La carpeta static contiene los recursos públicos y estáticos de la aplicación, necesarios para el diseño y la interactividad. Dentro de static, se encuentran subdirectorios como js, que almacena archivos de JavaScript para la funcionalidad del lado del cliente (estadisticas.js, notifications.js, y archivos de admin template como sb-admin-2.js). En una de las imágenes se aprecia que la carpeta static también contiene los directorios scss, vendor, css, e img, lo que confirma que contiene hojas de estilo, imágenes y otros recursos de terceros. La estructura de js incluye una subcarpeta demo con archivos especializados para demostrar la funcionalidad de gráficos y tablas (chart-area-demo.js, datatables-demo.js, etc.).
ya conociendo esta estructura

Esta organización es típica de un desarrollo MVC (Model-View-Controller), donde app.py actúa como controlador, templates como vistas, y los módulos de preprocesamiento manejan parte de la lógica del modelo de datos.
Conociendo y entendiendo la estructura del proyecto (la cual incluye el backend manejado por app.py, los templates HTML y los archivos estáticos), se procede ahora a mostrar el resultado visual de la aplicación: el dashboard. Este dashboard es la interfaz principal generada y servida por el script app.py, el cual procesa los datos, define las rutas y renderiza las diferentes vistas HTML (como index.html o estadisticas.html dentro de la carpeta templates) para presentar la información de manera gráfica y organizada al usuario. Es en este punto donde se visualiza el fruto del trabajo de preprocesamiento y la conexión de todos los componentes.

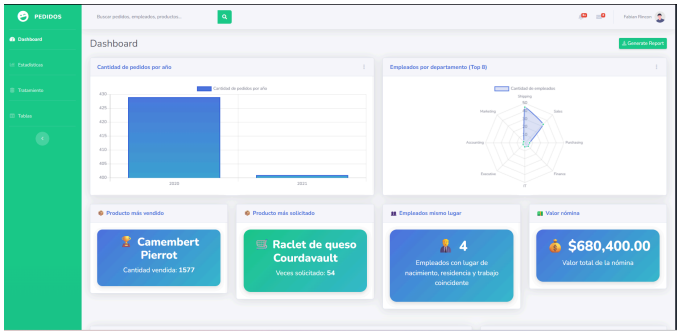

Fig. 5. Vista gráfica del Dashboard

La primera vista que se presenta al usuario corresponde a la pestaña principal, denominada "Dashboard". En esta sección, el usuario puede observar los resultados de las consultas y análisis de datos que fueron desarrollados en trabajos anteriores del curso. Es fundamental destacar que todos estos datos no se cargan directamente, sino que son previamente procesados y transformados mediante la lógica contenida en el archivo preprocesamiento.py (ubicado en la carpeta fpreprocesamiento). Este archivo garantiza que la información mostrada en el dashboard esté limpia, estructurada y lista para ser visualizada mediante gráficos y estadísticas resumidas.

La Consulta 1 tiene como objetivo principal determinar la cantidad de productos existentes por cada categoría dentro de la base de datos pedidos11. Para lograr esto, la función Python lee_archivo() establece una conexión a la base de datos y ejecuta una sentencia SQL que realiza una unión (LEFT JOIN) entre las tablas categorias y productos. Esto permite asociar cada producto con su respectiva categoría. Luego, el resultado se agrupa (GROUP BY) por el nombre de la categoría para contar (COUNT) cuántos productos están asignados a ella. Finalmente, la información se presenta ordenada de forma descendente por la cantidad de productos, y el resultado es cargado en un DataFrame de Pandas para su posterior procesamiento y visualización en el dashboard.

Una vez que los datos de la Consulta 1 han sido obtenidos y procesados en un DataFrame por el módulo de preprocesamiento, la aplicación requiere una interfaz (API) para servir estos datos a la capa de presentación (el frontend).

```python
# ==============================
# CONSULTA 1: Productos por categoría
# ==============================
def lee_archivo():
    conexion = pymysql.connect(
        host="127.0.0.1",
        port=3308,
        user="root",
        password="040616",
        database="pedidos11"
    )

    consulta = """
    SELECT
        c.Nombre_Categoria AS Categoria,
        COUNT(p.Id_Producto) AS Cantidad
    FROM categorias c
    LEFT JOIN productos p ON c.Id_Categoria = p.Id_Categoria
    GROUP BY c.Nombre_Categoria
    ORDER BY Cantidad DESC;
    """

    df = pd.read_sql(consulta, conexion)
    conexion.close()
    return df
```

Fig. 6. Consulta 1 hecha desde el archivo preprocesamiento.py

La función obtener_datos_dinamicos(df_emp) toma el DataFrame resultante de la consulta (que contiene las categorías y las cantidades de productos) y lo transforma en una estructura de datos compatible con JavaScript. Específicamente, extrae las columnas 'Categoria' y 'Cantidad' en listas separadas, las cuales se empaquetan en un diccionario junto con un título para el gráfico. El propósito de esta función es preparar la información en un formato que la librería de visualización Chart.js pueda consumir directamente. Posteriormente, la función datos_grafico(), decorada con @app.route("/api/datos-grafico"), establece un endpoint o ruta de API dentro de la aplicación Flask. Cuando el navegador o la interfaz de usuario realizan una solicitud fetch a esta URL, esta función ejecuta obtener_datos_dinamicos(), recibe el diccionario de datos y utiliza jsonify(datos) para enviarlo como una respuesta JSON al frontend. Esto permite que el

dashboard cargue dinámicamente el gráfico de "Cantidad de productos por categoría" al solicitar los datos a esta URL.

```python
# ====================================================
# API: Datos dinámicos para Chart.js
# ====================================================
def obtener_datos_dinamicos(df_emp):
    """
    Convierte el DataFrame (categorías y cantidades)
    en un formato JSON legible por Chart.js
    """
    print("Columnas del DataFrame:", df_emp.columns)

    # ✅ Usa los nombres reales de las columnas
    lista_categorias = df_emp['Categoria'].tolist()
    lista_cantidades = df_emp['Cantidad'].tolist()
    titulo = "Cantidad de productos por categoría"

    # Retorna en formato JSON
    datos = {
        'labels': lista_categorias,
        'data': lista_cantidades,
        'titulo': titulo
    }
    return datos


@app.route("/api/datos-grafico")
def datos_grafico():
    """
    Endpoint que devuelve los datos del gráfico
    en formato JSON (usado por Chart.js vía fetch)
    """
    datos = obtener_datos_dinamicos(df_emp)
    return jsonify(datos)
```

Fig. 7. API para Datos Dinámicos de la consulta 1

Una vez que el frontend recibe la respuesta JSON del endpoint /api/datos-grafico, el código de la interfaz de usuario, típicamente escrito en JavaScript, procede a consumir y utilizar estos datos para la visualización.

El objeto JSON contiene las listas de labels (las categorías de productos) y data (la cantidad correspondiente), que son los insumos directos que necesita la librería Chart.js para dibujar el gráfico. La aplicación utiliza el mecanismo de fetch o AJAX para solicitar estos datos de forma asíncrona.

Para darle el estilo visual y definir las opciones de presentación de la gráfica (como colores, tipo de gráfico, escalas, etc.), el código JavaScript se apoya en el archivo chart-area-demo.js. Este archivo de demostración, ubicado en la carpeta static/js/demo, contiene las configuraciones predefinidas del dashboard (probablemente provenientes de una plantilla administrativa) y es el responsable de tomar los datos dinámicos del JSON y combinarlos con las opciones de estilo para renderizar el gráfico de área final en el dashboard, asegurando una presentación coherente y atractiva de la información "Cantidad de productos por categoría".

```javascript
function inicializarGraficaCategorias() {
    fetch('/api/datos-grafico')
        .then(res => res.json())
        .then(datos => {
            console.log("✅ Gráfico 1 - Productos por categoría cargado");
            const ctx = document.getElementById("myAreaChart");
            if (!ctx) {
                console.error("❌ No se encontró el canvas 'myAreaChart'");
                return;
            }

            new Chart(ctx, {
                type: 'bar',
                data: {
                    labels: datos.labels,
                    datasets: [{
                        label: datos.titulo,
                        data: datos.data,
                        backgroundColor: [
                            'rgba(78, 115, 223, 0.6)',
                            'rgba(28, 200, 138, 0.6)',
                            'rgba(246, 194, 62, 0.6)',
                            'rgba(231, 74, 59, 0.6)',
                            'rgba(133, 135, 150, 0.6)',
                            'rgba(54, 185, 204, 0.6)',
                            'rgba(255, 193, 7, 0.6)',
                            'rgba(40, 167, 69, 0.6)'
                        ],
                        borderColor: 'rgba(78, 115, 223, 1)',
                        borderWidth: 1
                    }]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: {
                        yAxes: [{
                            ticks: { beginAtZero: true, precision: 0 }
                        }]
                    },
                    legend: { display: false }
                }
            });
        })
        .catch(error => {
            console.error("❌ Error en gráfico 1:", error);
        });
}
```

Fig. 8. Estilo de la grafía consulta 1 visto desde el archivo JavaScript

La función inicializarGraficaCategorias() es el componente de JavaScript encargado de realizar la carga y visualización del gráfico de "Productos por categoría" en el dashboard. Esta función inicia realizando una solicitud asíncrona (fetch) al endpoint de la API /api/datos-grafico para obtener los datos necesarios en formato JSON. Una vez que los datos son recibidos y procesados (.then(datos => ...)), la función localiza el elemento canvas en la página HTML con el ID myAreaChart, que servirá como contenedor para el gráfico. Finalmente, se instancia un nuevo objeto Chart (de la librería Chart.js). Esta instancia recibe los datos dinámicos (etiquetas en datos.labels y cantidades en datos.data) y aplica una configuración visual específica, incluyendo la definición de un gráfico de barras (type: 'bar'), una paleta de colores (backgroundColor) con valores RGBa para cada barra, y diversas opciones de presentación (options) como la capacidad de respuesta (responsive: true) y la configuración del eje Y para que comience en cero. Este proceso completa el ciclo, transformando los datos procesados en Python en una visualización interactiva y estilizada para el usuario.

Para mantener la concisión y la fluidez de este informe, es importante señalar que el proceso de creación del dashboard se replicó de manera idéntica para las 16 consultas restantes del proyecto.

Esto significa que, para cada consulta (como ventas por empleado, productos más vendidos, etc.), se siguió la misma estructura modular: primero, se definió y ejecutó la consulta SQL en el módulo de preprocesamiento; luego, se creó una función específica en el backend de app.py para transformar esos datos en formato JSON a través de una nueva ruta de API; y, finalmente, se desarrolló la función de JavaScript correspondiente para consumir ese JSON y renderizar el gráfico asociado en el dashboard, aprovechando la misma estilización y configuración base. De esta forma, se garantizó la consistencia y la eficiencia en la presentación de toda la información analítica.

Dentro de la interfaz principal del Dashboard, además de las visualizaciones de datos, se incorpora una sección de utilidad para el usuario, accesible a través del menú desplegable asociado al perfil del usuario (Fabian Rincon).

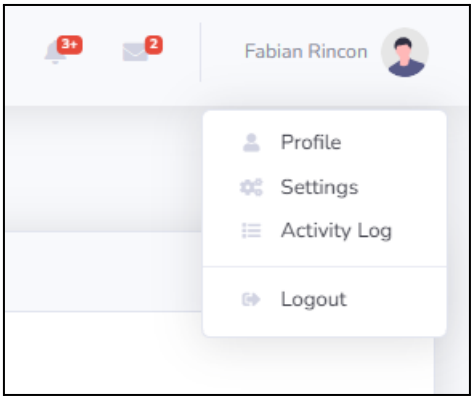

Fig. 9. Perfil de usuario

Como se aprecia en la figura 9, esta sección incluye opciones estándar de gestión de cuenta: Profile, Settings, Activity Log, y Logout. Específicamente, se ha configurado la funcionalidad de la pestaña Profile (profile.html en la estructura de templates). Esta vista permite a los usuarios visualizar o modificar la información personal que está registrada en el sistema. Aunque la imagen solo muestra el menú de navegación, su inclusión garantiza que la aplicación no solo sea un visor de datos analíticos, sino también una herramienta con capacidades básicas de gestión de usuario y autenticación.

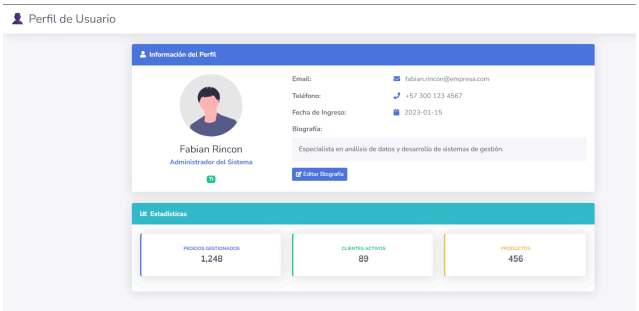

Fig. 10. Detalle del Perfil y Métricas de Gestión

La sección superior, etiquetada como "Información del Perfil", detalla los datos personales del usuario, en este caso, Fabian Rincon, identificado como "Administrador del Sistema". Se visualizan datos de contacto como el Email (fabian.rincon@empresa.com) y el Teléfono, junto con su Fecha de Ingreso (2023-01-15) y una Biografía personalizable que lo describe como "Especialista en análisis de datos y desarrollo de sistemas de gestión". El botón "Editar Biografía" sugiere una capacidad de interacción con esta información. Inmediatamente debajo, la sección "Estadísticas" presenta un resumen de las contribuciones o la actividad del usuario dentro del sistema, con tres métricas principales y de alto impacto.

Para lograr la creación de esta vista de "Perfil de Usuario" de manera funcional y dinámica, se empleó el archivo profile.html, ubicado en la carpeta templates, el cual define la estructura y el diseño visual de la página. Sin embargo, el contenido que se presenta, que incluye tanto la información personal del usuario (email, teléfono, biografía) como las métricas estadísticas clave (Pedidos Gestionados, Clientes Activos y Productos), se carga de forma completamente dinámica. Esto se consigue gracias a la invocación de las APIs y endpoints necesarios que han sido configurados en el archivo principal app.py. Estas APIs son responsables de consultar la base de datos y servir los datos actualizados a la interfaz, asegurando que el perfil no solo sirva como una tarjeta de presentación estática, sino también como un resumen ejecutivo de la actividad y la gestión del administrador dentro del sistema.

Otra sección fundamental de la aplicación es la pestaña de "Estadísticas", accesible desde el menú de navegación lateral. Esta vista está diseñada para ofrecer un análisis estadístico avanzado de los datos contenidos en la base de datos pedidos11.

La interfaz de esta sección se define mediante el archivo estadisticas.html, ubicado en la carpeta templates. Para la funcionalidad y el procesamiento de las visualizaciones específicas de esta pestaña, se utiliza el archivo estadisticas.js (ubicado en js/demo), que es el encargado de consumir las APIs del backend y renderizar los gráficos y tablas. Esta sección se organiza mediante pestañas interactivas para clasificar el tipo de análisis: Variables Cuantitativas, Variables Cualitativas, y Medidas de Dispersión.

Análisis de Variables Cuantitativas (Precios de Productos)

Esta subsección se centra en el análisis de variables numéricas, tomando como ejemplo los Precios de los Productos.

- Medidas de Tendencia Central y Rango: Se presentan indicadores clave como la Media ($28.87 - Promedio de precios$), la Mediana ($19.50 - Valor central$), la Moda ($14.00 - Valor más frecuente$), y el Rango ($261.00 - De $2.50 a 263.50).

- Distribución de Precios: Se visualiza la Distribución de Precios mediante un histograma o gráfico de barras agrupadas, mostrando la Cantidad de Productos dentro de diferentes intervalos de precio ($2-46, $46-89, etc.), lo que permite identificar rápidamente los rangos de precios con mayor concentración de productos.

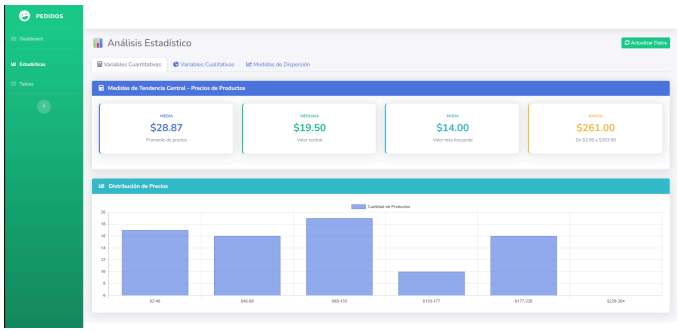

Fig. 11. Análisis de Variables Cuantitativas (Precios de Productos)

Análisis de Variables Cualitativas (Categorías y Formas de Pago)

El análisis de variables categóricas se ilustra con la Frecuencia de Categorías y las Formas de Pago.

- Frecuencia de Categorías: Se presenta una tabla de frecuencias que muestra la cantidad de productos por cada Categoría (ej. Repostería, Pescado/Marisco, Condimentos). La tabla incluye la Frecuencia absoluta y el Porcentaje de productos que corresponde a cada una, siendo Repostería la categoría más frecuente con un 16.90%.
- Formas de Pago: Se utiliza un Gráfico de Pastel (Dona) para visualizar la distribución de las Formas de Pago utilizadas en los pedidos, segmentando las transacciones entre Crédito y Efectivo para una rápida comprensión de la preferencia de pago.

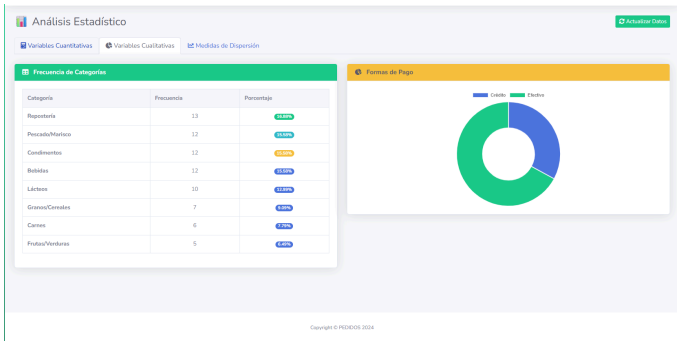

Fig. 12. Análisis de Variables Cualitativas (Categorías y Formas de Pago)

Esta pestaña se enfoca en la variabilidad de los datos, ejemplificada con el análisis de los Salarios de los Empleados.

- Medidas de Dispersión: Se cuantifica la dispersión con métricas como la Desviación Estándar ($3902.65 - Dispersión de salarios$), la Varianza ($15230688.23 - Variabilidad cuadrática$), y el Rango Intercuartílico (5650.00), mostrando los valores de $Q1$ y $Q3$ asociados.
- Diagrama de Caja (Box Plot): La dispersión se visualiza gráficamente mediante un Diagrama de Caja (Box Plot), representado aquí como barras. Este gráfico ilustra visualmente la distribución de los salarios, mostrando los puntos clave de la distribución: Mínimo, Q1, Mediana, Q3, y Máximo, lo cual es una herramienta poderosa para identificar la simetría, la dispersión y la presencia de posibles valores atípicos en los salarios.

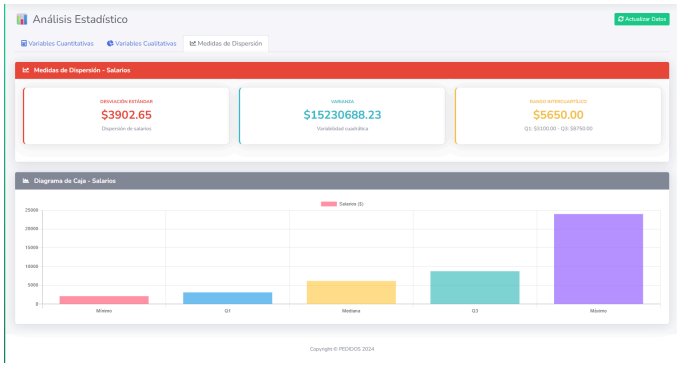

Fig. 13. Análisis de Medidas de Dispersión (Salarios de Empleados)

La siguiente sección clave del proyecto es la pestaña de "Tratamiento de Datos", diseñada para abordar las etapas fundamentales de la limpieza y preparación de datos antes del análisis.

La interfaz visual de esta sección se define en el archivo tratamiento.html dentro de la carpeta templates. La funcionalidad completa, que incluye la detección y aplicación de transformaciones, se logra mediante una estrecha colaboración entre las funciones de procesamiento definidas en preprocesamiento.py y los endpoints API configurados en app.py. Este enfoque permite que el usuario interactúe directamente con las etapas de limpieza y preparación de datos a través de una interfaz amigable.

El tratamiento de datos comienza con la identificación de problemas comunes en las bases de datos:

- 1. Valores Faltantes: Se detectan los valores nulos (NaN) en cada tabla. Se identifica, por ejemplo, que las tablas departments, employees, y lugares tienen columnas con registros faltantes (ej. MANAGER_ID, COMMISSION_PCT, Id_Lugar_Padre). La interfaz ofrece opciones para "Detectar Valores Nulos" y "Tratar Automáticamente" estos datos.
- 2. Valores Atípicos (Outliers): Se utiliza el método del Rango Intercuartílico (IQR) para detectar valores atípicos en variables clave como salarios, precios y cantidades. La detección muestra el porcentaje de

outliers y sus límites, por ejemplo, un 0.94% de outliers en salarios de empleados.

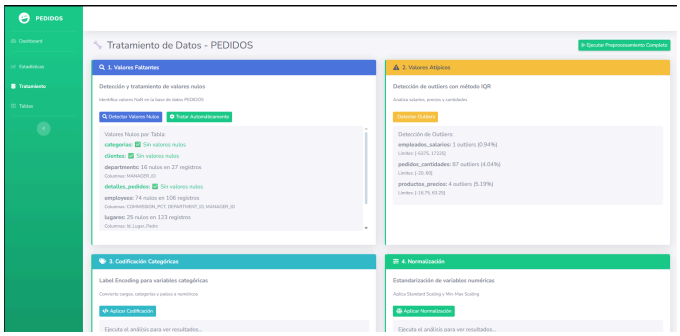

Fig. 14. Detección y Tratamiento (1 y 2)

Las siguientes etapas se enfocan en la conversión y estandarización de las variables:

- 3. Codificación Categóricas (Label Encoding): Este proceso convierte variables categóricas (como cargos o categorías de productos) en valores numéricos para que puedan ser utilizadas en modelos de aprendizaje automático. Aunque en la imagen se muestra un error (TypeError), la intención es aplicar esta codificación para que el sistema pueda procesar estas variables.
- 4. Normalización: Se aplica la Estandarización (Standard Scaling) y la Normalización Mín-Máx (Min-Max Scaling) a las variables numéricas (como precios y salarios). Esto asegura que todas las variables tengan la misma escala, lo cual es crucial para muchos algoritmos de machine learning. La interfaz muestra el valor original, el promedio (μ=0, σ=1) después de la estandarización y el rango [0, 1] después de la normalización Mín-Máx.

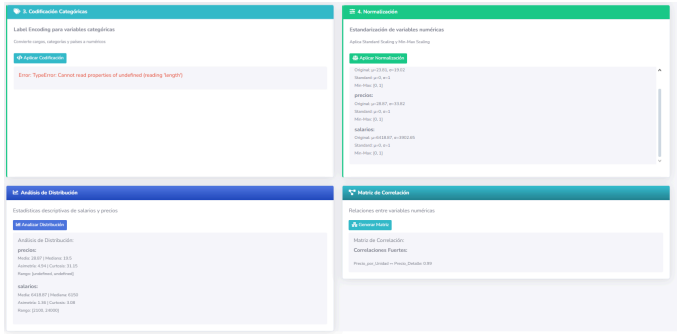

Fig. 15. Detección y Tratamiento (3 y 4)

Finalmente, la pestaña incluye herramientas de verificación para evaluar el impacto del tratamiento:

- Análisis de Distribución: Proporciona estadísticas descriptivas (Media, Mediana, Asimetría, Curtosis) de variables clave como salarios y precios, permitiendo al usuario entender cómo ha cambiado la distribución de los datos después de los pasos de limpieza y normalización.
- Matriz de Correlación: Permite "Generar Matriz" para identificar las relaciones fuertes entre las variables numéricas, como la correlación de 0.99 entre Precio_por_Unidad y Precio_Detalle. Este paso es fundamental para seleccionar características (Feature Selection) para modelos predictivos.

La sección final de la aplicación, y una de las más directas en términos de interacción con los datos, es la pestaña "Tablas".

Esta sección sirve como un navegador de la estructura de la base de datos pedidos11. Su interfaz principal está definida por el archivo tables.html (dentro de templates) y presenta un listado claro y conciso de todas las tablas utilizadas en el proyecto. Como se observa en la imagen, el usuario puede ver los nombres de las ocho tablas principales: CATEGORIAS, CLIENTES, DEPARTMENTS, DETALLES_PEDIDOS, EMPLOYEES, LUGARES, PEDIDOS, y PRODUCTOS.

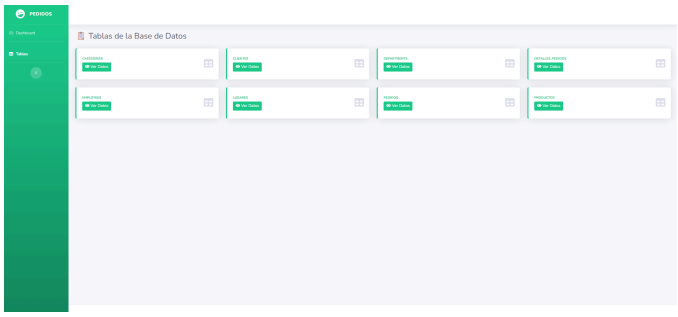

Fig. 16. Visualización Directa de las Tablas

Cada tabla tiene un botón "Ver Datos" que permite al usuario acceder al contenido completo de esa tabla. Al hacer clic en este botón, la aplicación redirige a una vista de detalle, la cual es gestionada por el template tabla_detalle.html. Este template se encarga de mostrar la información de la tabla seleccionada en un formato tabular paginado y legible. Esta funcionalidad permite la inspección directa de los datos sin necesidad de acceder a una herramienta de gestión de bases de datos externa, cerrando así el ciclo de la aplicación: desde la visualización de datos procesados (Dashboard/Estadísticas) hasta la revisión del dato crudo.

Descripción del Modelo E-R y Modelo Relacional

La base de datos pedidos11 fue diseñada siguiendo el Modelo Entidad-Relación (E-R) para representar de manera conceptual la información, y posteriormente se implementó bajo el Modelo Relacional utilizando SQL.

1. Modelo Entidad-Relación (E-R)

El modelo conceptual se basa en la identificación de las principales entidades y las relaciones que existen entre ellas, capturando la lógica del negocio de gestión de pedidos.

- Entidades Centrales: Las entidades principales son Pedidos, Clientes, Productos, y Employees.
- Relaciones Clave:
    - Un Pedido es realizado por un Cliente (relación 1:N).
    - Un Pedido es gestionado por un Employee (relación 1:N).
    - La relación M:N (muchos a muchos) entre Pedidos y Productos se resuelve mediante la tabla detalles_pedidos, que almacena los atributos de la transacción (cantidad, precio, descuento).

Otras entidades de soporte incluyen Categorias (para clasificar productos), Departments (para organizar empleados), y Lugares (para manejar ubicaciones geográficas de clientes y empleados).

2. Modelo Relacional

El modelo relacional se implementa a través de las tablas y el uso de claves primarias (PK) y claves foráneas (FK), cumpliendo con la Tercera Forma Normal (3NF) para evitar redundancia de datos.

- Esquema de Tablas: Se definieron ocho tablas principales: categorias, clientes, departments, employees, lugares, productos, pedidos, y detalles_pedidos.

- Integridad Referencial: La conexión lógica entre las tablas se garantiza mediante las restricciones de clave foránea (FOREIGN KEY), asegurando la integridad referencial. Por ejemplo

    - La tabla pedidos tiene Id_Cliente (FK) que referencia a clientes(Id_Cliente) y EMPLOYEE_ID (FK) que referencia a employees(EMPLOYEE_ID).
    - La tabla detalles_pedidos utiliza una clave compuesta formada por Id_pedido y Id_Producto, y ambas columnas son claves foráneas que enlazan con las tablas pedidos y productos, respectivamente.

Esta estructura relacional es la base sobre la cual se realizaron todas las consultas (SELECT... JOIN...) y el posterior preprocesamiento y análisis de datos en la aplicación.

Diagrama de bloques de los procesos

El diagrama de bloques ilustra de manera clara el flujo de datos y el control de procesos que se lleva a cabo para que la información analítica de la base de datos se muestre en el dashboard de la aplicación web. Este proceso sigue un camino lógico que abarca seis etapas interconectadas:

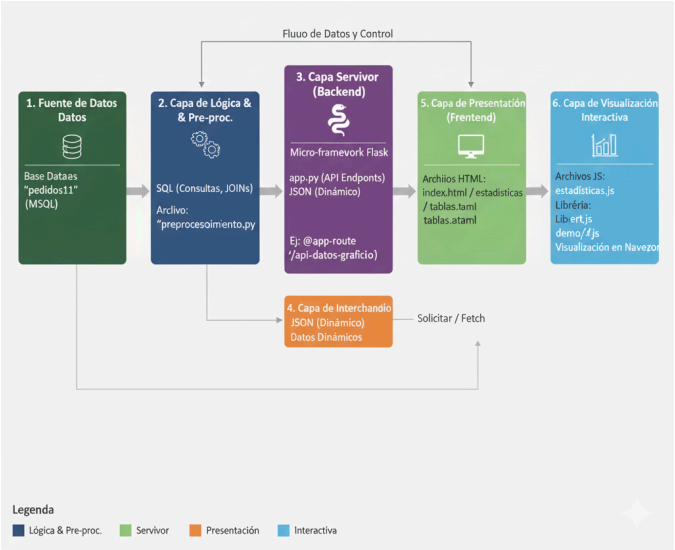

Fig. 17. Diagrama de bloques

1. Fuente de Datos y 2. Capa de Lógica y Preprocesamiento

El flujo comienza en la Fuente de Datos (1), donde la información reside en la Base de Datos pedidos11 (MySQL). La Capa de Lógica y Preprocesamiento (2), implementada en el archivo preprocesamiento.py, es la encargada de interactuar con esta fuente. Aquí se ejecutan las Consultas SQL (incluyendo JOINs y agregaciones) para extraer los datos crudos, realizar el tratamiento necesario (limpieza, normalización) y convertirlos en métricas útiles. El resultado de este procesamiento es una estructura de datos limpia, lista para ser servida.

3. Capa de Servidor (Backend) y 4. Capa de Intercambio

La Capa de Servidor (Backend) (3) está definida en el archivo principal app.py, que utiliza el micro-framework Flask. Este archivo cumple una función de control vital: recibe los datos procesados desde preprocesamiento.py y define los API Endpoints (rutas específicas como /api/datos-grafico). La función de Flask es transformar los datos en un formato estándar de intercambio web, lo que da lugar a la Capa de Intercambio (4), donde los datos analíticos se empaquetan en formato JSON para ser accesibles al frontend.

5. Capa de Presentación (Frontend) y 6. Capa de Visualización Interactiva

El frontend comienza con la Capa de Presentación (5), conformada por los archivos HTML (index.html,

estadisticas.html, etc.) ubicados en la carpeta templates. Estos archivos definen la estructura y la interfaz visual del dashboard. Cuando el navegador carga estas páginas, el código JavaScript (ubicado en la Capa de Visualización Interactiva (6), principalmente en estadisticas.js y archivos de demo/*.js) ejecuta una solicitud de datos (fetch) a los endpoints de la API definidos en app.py. Al recibir el JSON, el JavaScript lo consume y utiliza librerías como Chart.js para renderizar dinámicamente los gráficos y tablas en el navegador, completando así el ciclo de visualización de datos.

Conclusión

- El proyecto establece que la Base de Datos Relacional (pedidos11) es el punto de origen y la fuente de verdad de toda la información analítica. La correcta definición del Modelo Relacional, con sus Claves Primarias (PK) y Claves Foráneas (FK), garantiza la integridad referencial y la consistencia de los datos (pedidos, clientes, productos). Esta estructura organizada es lo que permite que el módulo de preprocesamiento en Python pueda ejecutar consultas SQL complejas con JOINs y agregaciones de manera eficiente, asegurando que las métricas visualizadas en el dashboard (ventas, stock, frecuencias) sean precisas y confiables. La calidad de la analítica es, por lo tanto, directamente dependiente de la solidez del diseño relacional inicial.

- La pestaña de "Tratamiento de Datos" resalta que los datos de la base de datos, aunque estructurados, no están inmediatamente listos para el análisis avanzado. El módulo preprocesamiento.py actúa como un puente crítico al aplicar técnicas estadísticas de limpieza (detección de outliers y nulos) y transformación (Normalización y Codificación Categórica) sobre los datos extraídos. Esto subraya el principio de que los sistemas de Bases de Datos deben ir acompañados de una capa de Data Wrangling para generar Business Intelligence (BI). La capacidad de normalizar variables como precios y salarios directamente desde la interfaz prepara los datos para futuros modelos de Machine Learning y asegura que las Estadísticas Descriptivas sean interpretables.

- La aplicación demuestra un flujo de trabajo moderno en el cual la Base de Datos no es accedida directamente por el frontend. En su lugar, el backend de Flask (app.py) gestiona la conexión a la base de datos y expone los resultados de las consultas a través de APIs (en formato JSON). Este mecanismo protege la base de datos de accesos directos, mejora la seguridad y asegura que la lógica de negocio (las reglas sobre cómo se consultan los datos) se mantenga en el servidor. Al utilizar el JSON como formato de intercambio, la aplicación logra una visualización dinámica y asíncrona en el dashboard, permitiendo la carga rápida y eficiente de múltiples gráficos sin recargar la página completa, optimizando la experiencia del usuario final.

Bibliografía

[1] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] D. K. Lin, SQL Server: The Complete Reference. New York, NY, USA: McGraw-Hill Education, 2012. (Referencia general para SQL y RDBMS, puedes sustituir por un libro específico de MySQL si lo usaste).

[3] M. Grinberg, Flask Web Development: Developing Web Applications with Python. Sebastopol, CA, USA: O'Reilly Media, 2018.

[4] P. S. Naidu, Programming Python: Powerful Object-Oriented Programming. India: CreateSpace Independent Publishing Platform, 2018.

[5] P. S. Naidu, Programming Python: Powerful Object-Oriented Programming. India: CreateSpace Independent Publishing Platform, 2018.

[6] R. C. T. Lee, "The History of JavaScript: Netscape's Bold Bet," IEEE Ann. Hist. Comput., vol. 27, no. 1, pp. 32-41, Jan.-Mar. 2005. (Referencia para el lenguaje JavaScript, usado en el frontend).